Shape Truthful Clustering (STC)
Algorithm Definitions

Prof. Dr. Asis Hallab

August 6, 2026

In this document we find the pseudocode definitions of the functions for the Shape Truthful Clustering (STC) variant tangent space.

1 Data Definitions

- Input multivariate vectors as an 2D Fortran real array.
- Logical mask defining the seeds.

2 Numerical Linear Algebra: SVD vs. Eigendecomposition

This module never forms a covariance or Gram matrix explicitly and eigendecomposes it. Forming $C = YY^\top$ (or $M^\top M$ for a comparison between two bases) squares the condition number of the underlying data relative to computing singular values of Y (or M) directly, which disproportionately degrades precision in the *small* eigenvalues – exactly the normal-space ones that `normal_error` and `tangent_scales` depend on. This is not a question of which eigensolver to use: even LAPACK's most robust symmetric eigensolver cannot recover precision already lost by forming the squared matrix in the first place. The fix is to never form it: compute singular values and vectors of the actual centered data (or comparison) matrix directly.

Two distinct problem shapes occur in this module, with two different recommended routines:

- **Full spectrum** (all D singular values/vectors of a $D \times k$ centered ensemble matrix $Y_{\mathcal{E}}$, needed by `observable`, `normal_error`, and `tangent_scales`): use `dgesdd` (divide-and-conquer SVD), economy mode (`JOBZ='S'`). It is the fastest LAPACK SVD routine for computing full singular vectors, and is standard/portable across any LAPACK/OpenBLAS build. It requires a genuine workspace *query* (call once with `LWORK=-1`, read the optimal size back from `WORK(1)`) rather than a closed-form minimum, plus an integer workspace array (`IWORK`, size $8 \cdot \min(D, k)$) – size both in the `_alloc` layer.
- **Small $d \times d$ matrix** (comparing two tangent bases, e.g. $M = U_{\mathcal{E}_t}^\top U_{\mathcal{E}_{t+1}}$, for principal angles in `accept_ensemble`): use `dgesvd`, not `dgesdd`. At d's typical size (the intrinsic dimension, usually 1-5), divide-and-conquer's speed advantage does not materialize and its extra `IWORK` bookkeeping is pure overhead for no benefit; `dgesvd`'s simpler single-workspace-array bookkeeping wins. Its singular values are $\cos\theta_j$ directly – no squaring, no `sqrt` needed back out.

Do not use `dsyev`, `dsyevd`, or `dsyevr` anywhere in this module. They solve "eigendecompose a given symmetric matrix as accurately as possible," which is not the problem this module has – it can choose not to form that symmetric matrix at all.

3 Functions / Subroutines

3.1 Coding / Naming convention

Follow `misc/Fortran_Coding_Guides.pdf` (the F42 standard) strictly, in particular section 10.2 ("Scientific Kernel (SK)") for SK purity/allocation rules and section 10.3.2 ("Naming Convention", p. 22) for the required suffix scheme:

- no suffix: pure SK routine (the numerical core; no I/O, no `allocate`/`deallocate`, deterministic);

- `_alloc`: allocation helper that computes required workspace sizes, allocates them, and calls the validated entry point below;
- `_C` / `_R`: C-/R-facing API wrappers;
- `_helper`: general-purpose orchestration helper not bound to a specific API.

The guide's suffix list has no separate "validate" suffix; input validation (via `src/macros.h`'s precompiler macros) lives in the *unsuffixed* routine, which validates its arguments and then calls the pure `_helper` core. This three-tier `_alloc` -> unsuffixed validated entry -> `_helper` pattern is already used on this branch in `src/tox_shatter_cluster_data.F90` (`calculate_density_radius_alloc` -> `calculate_density_radius` -> `calculate_density_radius_helper`); follow that same shape here rather than introducing a new naming scheme.

Related routines following this pattern are called SK-groups (SKGs).

3.2 Seeding

Seeding requires two dedicated SKGs. One to find the radius used to measure local density centered on a vector $\vec{x}_i$, the other to identify seeds.

The SKG `calculate_density_radius` to identify the radius receives a percentile as optional input (default 15%). The algorithm finds the mean vector $x = \frac{1}{N} \cdot \sum_{i=1}^{N} x_i$ with $x_i \in$ data_vectors Next, all distances are computed to this mean vector. Finally, the argument percentile distance is returned as the radius to be used in density label calculation.

A `seeds` SKG is used with input:

- data_vectors - 2D real array output:
- logical mask defining the selected seed points.

3.2.1 Algorithm

First, assign each $x_i \in$ data_vectors a density label. Use a or reuse the function `density_labels` for this that populates a 2D real array of length `n-columns(data_vectors)`.

Sort the density labels descending. Start with the first and identify all input data vectors within its radius. Mask those as visited. Continue with the next highest non visited density label until none remain.

3.3 Cluster identification

In parallel grow ensembles around each seed vector. Use OMP paradigms if and only if `do concurrent` parallelism is not supported due to external library calls (gfortran).

The SKG is called `cluster_identification` and is an iteration wrapper for the below steps.

Consider each seed an ensemble of size one. Growing is done iteratively until stop conditions are reached. Each iteration has an ensemble $\mathcal{E}_t$, an 2D array of o observables $\mathcal{O}_{t-o+1}, \mathcal{O}_{t-o+2}, \ldots, \mathcal{O}_t$.

3.3.1 First growth step

A seed is a single point: its `observable` would be the singular value decomposition of a one-column centered matrix, which is degenerate – no meaningful U, d, or G exists to compare against. `cluster_identification` therefore performs its first `grow_ensemble` + `observable` call unconditionally, without invoking `accept_ensemble`; there is nothing yet to compare the result against. `accept_ensemble` is invoked starting from the second growth iteration onward, comparing iteration 2's observable against iteration 1's.

This bootstrap rule lives here, in the iteration wrapper, and not in `seeds`, `grow_ensemble`, `observable`, or `accept_ensemble` themselves. `seeds`' purpose is identifying starting points, nothing else – and it runs *before* `calc_ensemble_growth_radius`, so it has no growth radius available to grow by even if it wanted to. Keeping `grow_ensemble`, `observable`, and `accept_ensemble` fully general and iteration-unaware keeps each independently testable in isolation. `cluster_identification` is the one place that already knows which iteration it is, without introducing any new state elsewhere.

3.3.2 Output

`cluster_identification` returns, per seed:

- `final_ensemble_mask(n_vectors)`, logical – the accepted ensemble's membership, using the same logical-mask convention as `seeds`.
- The trailing observable history, the last (up to) o iterations. Each $\mathcal{O}_t$ is not a flat vector – U is a $D \times d_t$ matrix whose width d_t can itself change between iterations, which is exactly what δ_d checks. Storing the **full** $D \times D$ decomposition per retained iteration, as already decided for `observable` above, sidesteps the variable-width problem: slice $U(:, 1{:}d_t)$ out on demand instead of needing variable-shape storage. Concretely:
  - `U_history(D, D, o)`, real;
  - `S_history(D, o)`, real – the singular values; eigenvalues, `normal_error`, and `tangent_scales` at any retained iteration are derived on demand from these plus `k_history` below, via the `normal_error`/`tangent_scales` SKGs, rather than stored redundantly;
  - `d_history(o)`, integer;
  - `G_history(o)`, real;
  - `mu_history(D, o)`, real;
  - `k_history(o)`, integer – ensemble size per retained iteration, needed to recover eigenvalues from singular values, $\lambda_j = s_j^2/(k-1)$.
- Optionally, `member_added_at_step(n_vectors)`, integer – 0 (or a documented sentinel) for non-members, the seed's own designation for the seed itself, and the growth-iteration index at which each subsequent member joined otherwise. Answers "which members were added when" and "did this ensemble oscillate during growth" directly.

3.3.3 Local Radius Identification

For growing ensembles we need a locally adapted radius with which at each step candidates for the addition are identified. Those candidates are vectors within the local ensemble specific radius.

The SKG should be called `calc_ensemble_growth_radius` or similar.

Use a fixed-count kNN pool and compute the median distance among a seed's own $k_{\min}$ nearest neighbors. Make $k_{\min}$ an optional argument with default value 30.

For each seed we store the growth radius in a 1D real array called `ensemble_growth_radii`.

3.3.4 Tangent Space Variant

This section describes the implementation used for the tangent space variant of STC.

Each iteration of `cluster_identification` runs:

- `grow_ensemble`
- `observable`
- `accept`

3.3.4.1 SKG `grow_ensemble` Identify the candidate points within the input ensemble's growth-radius $r_{\mathcal{E}}$:

$$\mathcal{C}_{\mathcal{E}_{t+1}} = \{x_k \in \text{data\_vectors} \mid ||x_k - x_i|| \leq r_{\mathcal{E}} \ \exists x_i \in \mathcal{E}\}$$

3.3.4.2 SKG `observable` Calculate the tuple $\mathcal{O}_{\mathcal{E}_{t+1}} = (U_{\mathcal{E}_{t+1}}, d_{\mathcal{E}_{t+1}}, G_{\mathcal{E}_{t+1}}, \boldsymbol{\mu}_{\mathcal{E}_{t+1}}, \text{normal\_error}_{\mathcal{E}_{t+1}}, \text{tangent\_scales}_{\mathcal{E}_{t+1}})$ for the input ensemble $\mathcal{E}_{t+1}$, where

(1) $U_{\mathcal{E}_{t+1}}$, the tangent basis, is obtained from the **singular value decomposition** of the Ensemble's centered member vectors – not from an eigendecomposition of their covariance (see "Numerical Linear Algebra" above for why). To obtain it, compute the neighborhood center

$$\boldsymbol{\mu}_{\mathcal{E}_{t+1}} = \frac{1}{|\mathcal{E}_{t+1}|} \sum_{x_j \in \mathcal{E}_{t+1}} \mathbf{x}_j,$$

construct the centered matrix

$$Y_{\mathcal{E}_{t+1}} = [\mathbf{x}_1 - \boldsymbol{\mu}_{\mathcal{E}_{t+1}}, \ldots, \mathbf{x}_{\mathcal{N}} - \boldsymbol{\mu}_{\mathcal{E}_{t+1}}], \text{for } x_1, \ldots, x_{\mathcal{N}} \in \mathcal{E}_{t+1},$$

and its economy-mode singular value decomposition, using LAPACK `dgesdd` (`JOBZ='S'`), over the **full** set of D singular values/vectors, not only the top-d tangent ones:

$$Y_{\mathcal{E}_{t+1}} = U_{\mathcal{E}_{t+1}} S_{\mathcal{E}_{t+1}} V_{\mathcal{E}_{t+1}}^{\top}.$$

The retained "leftover" pairs are what make center, normal error, and tangent scales free below, and are also exactly what a later noise-tube covariance $\Sigma_{\perp}$ (needed once ensembles are reconciled) would be built from.

The corresponding eigenvalues are recovered from the singular values as

$$\lambda_j^{\mathcal{E}_{t+1}} = \frac{\left(s_j^{\mathcal{E}_{t+1}}\right)^2}{k_{\mathcal{E}_{t+1}} - 1}, \qquad \lambda_1^{\mathcal{E}_{t+1}} \geq \lambda_2^{\mathcal{E}_{t+1}} \geq \cdots \geq \lambda_D^{\mathcal{E}_{t+1}} \geq 0.$$

LAPACK SVD routines already return singular values in *descending* order (unlike `dsyev`'s ascending eigenvalues), so this matches the document's own convention directly: the tangent basis and its eigenvalues are the *first* d columns/entries of `dgesdd`'s output, and the normal-space basis and eigenvalues are the remaining $D - d$.

(1b) $\boldsymbol{\mu}_{\mathcal{E}_{t+1}}$, the ensemble center, is simply the mean vector already computed above to form $Y_{\mathcal{E}_{t+1}}$ – no further work needed.

(1c) normal_error$_{\mathcal{E}_{t+1}}$ and tangent_scales$_{\mathcal{E}_{t+1}}$ are obtained from the retained eigenvalues via the `normal_error` and `tangent_scales` SKGs defined below.

(2) The Spectral Gap is calculated as follows.

For every candidate intrinsic dimension

$$r = 1, \ldots, D - 1$$

compute its Spectral Gap

$$G(r) = \frac{\lambda_r^{\mathcal{E}_{t+1}}}{\lambda_{r+1}^{\mathcal{E}_{t+1}} + \epsilon},$$

where ϵ is a minimal positive real number to avoid division by zero.

(3) and the estimated intrinsic dimension

$$d_{\mathcal{E}_{t+1}} = \arg\max_r G(r)$$

and set the current ensemble's spectral gap to

$$G_{\mathcal{E}_{t+1}} = G(d_{\mathcal{E}_{t+1}})$$

Note that an optional argument o with default value of e.g. 10 indicates how many observables of the last o iterations are stored. Thus, each ensemble $\mathcal{E}_{t+1}$ has a two dimensional real array of observables, one per column:

$$\Omega_{\mathcal{E}_{t+1}} = [\mathcal{O}_{\mathcal{E}_{t-o+2}}, \mathcal{O}_{\mathcal{E}_{t-o+3}}, \ldots, \mathcal{O}_{\mathcal{E}_{t+1}}]$$

3.3.4.3 SKG `normal_error` Given intrinsic dimension $d_{\mathcal{E}}$ and the eigenvalues $\lambda_1^{\mathcal{E}} \geq \cdots \geq \lambda_D^{\mathcal{E}}$ from `observable`, the mean squared residual of the ensemble's members off its $d_{\mathcal{E}}$-dimensional tangent subspace is

$$\text{normal\_error}_{\mathcal{E}} = \sum_{j=d_{\mathcal{E}}+1}^{D} \lambda_j^{\mathcal{E}}.$$

No pass over the ensemble's member vectors is required; the sum is already implied by the singular value decomposition computed in `observable`.

3.3.4.4 SKG `tangent_scales` Given the same $d_{\mathcal{E}}$ and eigenvalues, the extent along each tangent direction is

$$\text{tangent\_scales}_{\mathcal{E}}(j) = \sqrt{\lambda_j^{\mathcal{E}}}, \qquad j = 1, \ldots, d_{\mathcal{E}}.$$

3.3.4.5 SKG `accept_ensemble` This routine returns a Boolean indicating whether the grown ensemble $\mathcal{E}_{t+1}$ is accepted or not.

A grown ensemble ($\mathcal{E}_{t+1}$) is assessed based on its recent trajectory of observables $\Omega_{\mathcal{E}_{t+1}}$.

Acceptance is calculated comparing $\mathcal{O}_{\mathcal{E}_{t+1}}$ with $\mathcal{O}_{\mathcal{E}_t}$, and $\mathcal{E}_{t+1}$ is accepted if:

- principal angles $\alpha_i \leq \boldsymbol{\alpha}_{\max}, i = 1, \ldots, \min(d_{\mathcal{E}_{t+1}}, d_{\mathcal{E}_t})$ between $U_{\mathcal{E}_{t+1}}$ and $U_{\mathcal{E}_t}$ – obtained via `dgesvd` on $M = U_{\mathcal{E}_t}^{\top} U_{\mathcal{E}_{t+1}}$, whose singular values are $\cos \alpha_i$ directly (see "Numerical Linear Algebra" above); check $d_{\mathcal{E}_{t+1}} = d_{\mathcal{E}_t}$ first and short-circuit to reject on mismatch, since that is cheaper than the SVD and the ensembles do not share a common tangent dimension to compare angles over in that case, and
- $|d_{t+1} - d_t| \leq \boldsymbol{d}_{\max}$, and
- $|log \frac{G_{\mathcal{E}_{t+1}}}{G_{\mathcal{E}_t}}| \leq \boldsymbol{G}_{\max}$